
TECHNICAL HANDOFF DOCUMENT

TENNIS CLUB AMBROSIANO WEBSITE — TECHNICAL DOCUMENTATION

A complete reference to the codebase, architecture, content model, integrations and deployment pipeline of tcambrosiano.com — written for a new development agency taking over maintenance and future development.

PROJECT	Tennis Club Ambrosiano — public website (tcambrosiano.com)
REPOSITORY	github.com/maurizio-gif/WebSite-TCA
STACK	Astro 5 (static site), pure CSS, TinaCMS (headless CMS, scaffolded)
HOSTING	Vercel (production) · GitHub Pages (preview)
DOCUMENT SCOPE	Architecture, content model, components, integrations, SEO, deployment, CI/CD, maintenance playbook
DOCUMENT DATE	August 2026 (rev. 2 — third-party links centralized; lead-data downstream flow documented)

TABLE OF CONTENTS

1.	Introduction & Purpose
2.	Technology Stack
3.	Repositories, Branches & Environments
4.	Local Development Setup
5.	Project Structure
6.	Routing Model & Page Inventory
7.	Internationalization (i18n)
8.	Content Model — Astro Content Collections
9.	TinaCMS — Headless CMS
10.	Global Layout & Chrome
11.	Component Reference
12.	Lead Capture & Forms System
13.	Third-Party Integrations
14.	SEO Implementation
15.	Styling System
16.	Assets & Media Conventions
17.	Content Editing Conventions
18.	Deployment & CI/CD
19.	Environment Variables & Secrets
20.	Known Conventions, Constraints & Gotchas
21.	Maintenance Playbook

APPENDICES

- Full Route Table (IT / EN)
- Full Component Index
- Content Collections — Field Reference
- Third-Party Endpoints & External Links

1. INTRODUCTION & PURPOSE

This document describes the tcambrosiano.com website end-to-end: what it is built with, how the codebase is organized, how content is authored and translated, which third-party services it talks to, and how it is built, previewed and deployed. It is written so that a development team with no prior exposure to the project can pick it up, understand every moving part, and safely maintain or extend it.

The site is a **statically generated** marketing and membership website for a sports club (tennis, padel, pickleball, athletic training, a junior tennis academy and the historic "Torneo Avvenire" youth tournament) based in Milan, Italy. It is bilingual (Italian/English), integrates a headless CMS for non-technical content editing, and funnels visitors into a shared lead-capture form that talks to an external marketing-automation backend (n8n) and to Google Ads/Analytics via Google Tag Manager.

HOW TO USE THIS DOCUMENT

Sections 1–9 cover architecture and content; sections 10–17 cover the front-end implementation in depth; sections 18–20 cover operations (deployment, secrets, gotchas); section 21 is a task-oriented playbook for the most common maintenance requests. The appendices are quick-reference tables — use them once you already understand the concepts from the numbered sections.

The project also ships an internal `CLAUDE.md` file at the repository root, which is the canonical, machine-readable convention document used by AI coding assistants (and a useful quick-reference for humans too): stack, palette, directory layout and house rules. This document expands on it with full technical depth; where the two differ, treat `CLAUDE.md` as the living source of truth for conventions, and this document as the deeper architectural reference.

2. TECHNOLOGY STACK

LAYER	CHOICE	NOTES
Framework	Astro 5 (<code>astro@^5.0.0</code>)	Static output (<code>astro build → dist/</code>). No SSR, no client-side framework (no React/Vue/Svelte). Pages are <code>.astro</code> files compiled to static HTML at build time.
Styling	Plain CSS + CSS custom properties	No Tailwind, no CSS framework, no CSS-in-JS. Every component owns a scoped <code><style></code> block; a small global stylesheet defines shared tokens and utilities.
Content	Astro Content Collections (<code>astro:content</code>)	Markdown files with Zod-validated frontmatter, defined in <code>src/content/config.ts</code> .
Headless CMS	TinaCMS (<code>tinacms@^2.0.0</code> , <code>@tinacms/cli</code>)	Scaffolded and fully configured; requires TinaCloud credentials to activate the visual editor (see §9).
Client-side JS	Vanilla JS/TS	No bundling framework beyond Astro's own; interactive widgets (menus, forms, lightbox, calendar) are hand-written <code><script></code> blocks or small shared modules under <code>src/lib/</code> .
Sitemap	<code>@astrojs/sitemap</code>	i18n-aware sitemap generation.
Markdown processing	<code>remark-breaks</code>	Single newlines in markdown become <code>
</code> instead of being collapsed — matters for content typed in Tina's editor.
Type checking	TypeScript 5 + <code>@astrojs/check</code>	<code>npm run check</code> runs <code>astro check</code> .
Package manager	npm	<code>package-lock.json</code> is committed.
Node version	20	Pinned in both GitHub Actions workflows.

WHAT THE STACK DELIBERATELY EXCLUDES

- **No server/database.** The site is 100% static HTML/CSS/JS at deploy time; there is no application server, no API routes, no database.
- **No client-side framework.** Interactivity is plain DOM scripting; there is no hydration model to reason about.
- **No CSS framework or component UI library** (explicit project rule — see `CLAUDE.md`).
- **No package for forms/analytics/consent** — the lead form, Google Consent Mode bridge, and UTM/visitor tracking are all hand-rolled (see §12–13).

3. REPOSITORIES, BRANCHES & ENVIRONMENTS

3.1 SOURCE CONTROL

Single GitHub repository: `github.com/maurizio-gif/WebSite-TCA` .

- `main` — production branch. Every push triggers both the Vercel production deploy and the GitHub Pages preview deploy (see §18).
- `feature/<name>` — convention for in-progress work (per `CLAUDE.md`). Pull requests into `main` run the CI workflow (type check, build, redirect validation) before merge.

3.2 HOSTING ENVIRONMENTS

ENVIRONMENT	PLATFORM	URL	TRIGGER	BUILD COMMAND
Production	Vercel	tcambrosiano.com	Push to <code>main</code> (Vercel Git integration)	<code>npm run tina:build → tinacms build --skip-cloud-checks && astro build ,output dist/</code>
Preview / secondary	GitHub Pages	maurizio-gif.github.io/WebSite-TCA/	Push to <code>main</code> (<code>.github/workflows/deploy.yml</code>)	<code>npm run build</code> with <code>GITHUB_PAGES=true</code>

The two environments differ in one important structural way: on GitHub Pages the site is served from a **subpath** (`/WebSite-TCA/`), because GitHub Pages serves project sites from `username.github.io/repo-name/` . On Vercel the site is served from the domain root. This is handled centrally in `astro.config.mjs` and `src/i18n/utlis.ts` — see §7 and §18.

DO NOT HARDCODE ABSOLUTE INTERNAL LINKS OR IMAGE PATHS

Because of the GitHub Pages subpath, every internal link and every reference to a file in `public/` must go through the `getLocalePath()` / `publicUrl()` helpers in `src/i18n/utlis.ts` , never a raw `/path` string. A hardcoded path works fine on Vercel (root-served) but silently 404s on the GitHub Pages preview.

4. LOCAL DEVELOPMENT SETUP

4.1 REQUIREMENTS

- Node.js 20.x
- npm

4.2 INSTALL & RUN

```
git clone https://github.com/maurizio-gif/WebSite-TCA.git
cd WebSite-TCA
npm install
npm run dev          # http://localhost:4321 – Astro dev server, hot reload
```

4.3 NPM SCRIPTS

SCRIPT	COMMAND	PURPOSE
dev / start	astro dev	Local dev server with hot reload.
tina:dev	tinacms dev -c "astro dev"	Dev server with the TinaCMS visual editor active (needs <code>TINA_CLIENT_ID</code> / <code>TINA_TOKEN</code> to fully authenticate — see §9).
build	astro build	Production build → <code>dist/</code> . Used by the GitHub Pages workflow and by CI.
tina:build	tinacms build --skip-cloud-checks && astro build	Vercel's build command — generates Tina's admin bundle, then builds the site.
preview	astro preview	Serve the built <code>dist/</code> locally, to sanity-check a production build.
check	astro check	TypeScript/Astro type checking (also run in CI).
check:redirects	node scripts/check-vercel-redirects.mjs	Validates <code>vercel.json</code> redirect rules (see §18.4). Also run in CI, twice.

There is no test suite (unit/e2e) in the repository at the time of writing. Quality gates are: TypeScript checking, a full production build, and the custom redirect-safety script — all three run in CI on every PR and every push to `main`.

5. PROJECT STRUCTURE

```
WebSite-TCA/
├─ astro.config.mjs      Astro config: site URL, i18n, sitemap, markdown plugins
├─ vercel.json           Vercel build config + ~60 legacy-URL redirect rules
├─ tsconfig.json         Extends astro/tsconfigs/base
├─ package.json          Scripts & dependencies
├─ CLAUDE.md             Project conventions (source of truth, see §1)
├─ .github/workflows/
│   └─ ci.yml            Type check + build + redirect validation (PRs & main)
│   └─ deploy.yml        GitHub Pages deploy (push to main)
├─ scripts/
│   └─ check-vercel-redirects.mjs  Static + build-aware validator for vercel.json
├─ docs/                Internal notes / historical briefs (not build input)
├─ public/              Static files served as-is: images (.avif/.jpg pairs),
│                       logos/, legal/ (PDFs), email/ (assets for external
│                       transactional emails), robots.txt
├─ src/
│   └─ assets/images|logos  Target folder structure for curated media (see CLAUDE.md);
│                           currently placeholders (.gitkeep) – actual images used by
│                           pages live flat in public/, not here (see §16)
│   └─ components/         One .astro file per component, PascalCase (see §11)
│   └─ content/
│       └─ config.ts        Zod schemas for every content collection (see §8)
│   └─ content/<collection>/*.md  The actual content files (see §8)
│   └─ data/               Small typed constants: bank details, help-desk categories,
│                           phone country-code list
│   └─ i18n/               it.ts / en.ts (UI string dictionaries) + utils.ts (see §7)
│   └─ layouts/
│       └─ BaseLayout.astro Document shell: head/meta/SEO/scripts, header/footer chrome
│   └─ lib/                Shared vanilla-JS logic: lead form state machine, markdown
│                           helpers for plain-text translated fields
│   └─ pages/              Italian routes (default locale, unprefixed)
│       └─ en/              English routes (mirrored 1:1, prefixed /en/)
│   └─ styles/
│       └─ global.css       Design tokens, resets, utilities, animation system
└─ tina/
    └─ config.ts            TinaCMS schema: collections + field UI, one template per
                            page under the "pagine" collection (see §9)
    └─ __generated__/       Auto-generated by the Tina CLI, do not hand-edit
```

6. ROUTING MODEL & PAGE INVENTORY

6.1 HOW ROUTING WORKS

Astro uses **file-based routing**: every `.astro` file under `src/pages/` becomes a route at build time. This project does *not* rely on Astro's built-in i18n routing to generate localized pages automatically. Instead, **each Italian page has a hand-written English twin** under `src/pages/en/`, importing the same layout and components, reading the same content-collection entry, and falling back field-by-field to Italian when no English translation exists (the `d.field_en ?? d.field` pattern — see §7.3 and the `tennis.astro / en/tennis.astro` pair as a reference implementation).

Astro's `i18n` config block in `astro.config.mjs` is set with `prefixDefaultLocale: false`, which simply tells Astro/the sitemap integration that Italian is unprefixed and English lives under `/en/` — it does not generate or translate pages by itself. All actual duplication is manual, file by file.

CONSEQUENCE FOR MAINTENANCE

Adding or changing a page means touching **two files** (the IT page and its `en/` twin) that are structurally identical except for the `lang` value, the `_en` field fallbacks, and any hardcoded English/Italian copy inline in the page (e.g. CTA labels not stored in content). See the Maintenance Playbook (§21.1) for the exact steps.

6.2 DYNAMIC ROUTES

FILE	GENERATES
<code>src/pages/eventi/[slug].astro (+ en/ twin)</code>	One page per entry in the <code>eventi</code> collection
<code>src/pages/news/[slug].astro (+ en/ twin)</code>	One page per entry in the <code>news</code> collection
<code>src/pages/club-life/help-desk/[slug].astro (+ en/ twin)</code>	One page per entry in the <code>helpdesk</code> collection

The full route inventory (every static and dynamic route, in both languages) is listed in [Appendix A](#).

7. INTERNATIONALIZATION (I18N)

The site supports Italian (default) and English. Translation happens on **two independent layers** that work together:

7.1 UI-STRING LAYER — SRC/I18N/

FILE	ROLE
<code>it.ts</code>	Nested dictionary of every static UI string (nav labels, buttons, form copy, footer, modals...). Exports <code>type UI = typeof it</code> .
<code>en.ts</code>	Same shape as <code>it.ts</code> , English strings.
<code>utils.ts</code>	The i18n toolkit — see below.

`utils.ts` exports:

- `getLangFromUrl(url)` — detects `it` / `en` from the URL path (accounting for the GitHub Pages base path).
- `useTranslations(lang)` — returns the matching dictionary object.
- `getLocalePath(lang, path)` — builds an **href-ready** path for a given language, including the base prefix (e.g. `IT /tennis` → `/WebSite-TCA/tennis` on GH Pages, English adds `/en`).
- `publicUrl(path)` — prefixes a `public/` asset path with the base path.
- `getAlternatePath(pathname)` — given a clean pathname, returns both the IT and EN equivalents, used to build the language switcher and `hreflang` tags.

Every component that renders text or links accepts an optional `lang` prop (default `'it'`), calls `useTranslations(lang)`, and builds links with `getLocalePath`. This is why English pages pass `lang="en"` explicitly into every shared component they use.

7.2 CONTENT-TRANSLATION LAYER — COLLECTION SCHEMAS

Every field in `src/content/config.ts` that holds editorial copy has a sibling optional field with an `_en` suffix (e.g. `title` / `title_en`, `hero_titolo` / `hero_titolo_en`). English pages read `d.field_en ?? d.field`: if a translator has not filled in the English variant yet, the Italian copy is shown instead of a blank field. This means the English site never has empty content — it silently degrades to Italian per-field until someone provides a translation, which is deliberate and lets content owners translate incrementally through TinaCMS without ever “breaking” the English site.

7.3 RICH TEXT VS. PLAIN TEXT ASYMMETRY

Because TinaCMS's `rich-text` field type does not have an official plain-string equivalent, the Italian body of long-form editorial collections (events, news, help-desk guides) uses Tina's rich-text editor (rendered via Astro's `<Content />`), while their English counterpart (`corpo_en`) is a plain multi-line string field, converted to HTML by `src/lib/richtext.ts` (`mdParagraphsHtml()`): blank line = new paragraph, single newline = `<br>`, **bold** and `[text](url)` links supported. This is intentional and documented inline in the schema — do not “fix” the asymmetry by trying to make English rich-text, it would require restructuring the Tina schema.

7.4 ADDING A THIRD LANGUAGE (IF EVER NEEDED)

- Add the locale to `i18n.locales` in `astro.config.mjs` and to the sitemap integration's `i18n.locales` map.
- Add a new dictionary file in `src/i18n/` and extend the `Lang` union and `translations` map in `utils.ts`.
- Extend `getLangFromUrl`, `getLocalePath` and `getAlternatePath` to handle a third path segment.
- Duplicate every page under a new `src/pages/<locale>/` folder.
- Add `<locale>` fields to every collection schema in `src/content/config.ts` and mirror them in `tina/config.ts`.

This is a significant, mechanical but wide-reaching change — budget accordingly if ever requested.

8. CONTENT MODEL — ASTRO CONTENT COLLECTIONS

All editorial content lives as Markdown files under `src/content/<collection>/` , validated against Zod schemas in `src/content/config.ts` , and read at build time with Astro's `getCollection()` / `getEntry()` . There is no runtime database — all content is baked into the static build.

8.1 COLLECTIONS OVERVIEW

COLLECTION	PATH	CARDINALITY	PURPOSE	CONSUMED BY
<code>pagine</code>	<code>src/content/pagine/*.md</code>	One file per route	SEO fields + hero + page-specific body sections for every page on the site	Every <code>src/pages/*.astro</code> file
<code>eventi</code>	<code>src/content/eventi/*.md</code>	One per event	Tournaments, clinics, camps shown on the homepage & the events page	<code>EventiHome.astro</code> , <code>eventi/[slug].astro</code>
<code>news</code>	<code>src/content/news/*.md</code>	One per article	"Club Life" news carousel/board	<code>ClubLifeNews.astro</code> , <code>news/[slug].astro</code>
<code>helpdesk</code>	<code>src/content/helpdesk/*.md</code>	One per guide	Club Life knowledge-base articles	<code>ClubLifeHelpDesk.astro</code> , <code>help-desk/[slug].astro</code>
<code>servizi</code>	<code>src/content/servizi/*.md</code>	One per service	"Services & Partners" cards on Club Life	<code>ClubLifeServizi.astro</code>
<code>planning</code>	<code>src/content/planning/corsi.md</code>	Singleton	Weekly group-class schedule	<code>WeeklySchedule.astro</code>
<code>membership</code>	<code>src/content/membership/membership.md</code>	Singleton	Silver/Gold/Platinum pricing table, shared across pages	<code>MembershipSection.astro</code>
<code>info</code>	<code>src/content/info/club.md</code>	Singleton	Address, hours, phone, email — shown site-wide	<code>InfoPratiche.astro</code>
<code>legal</code>	<code>src/content/legal/documenti.md</code>	Singleton	Two legal PDFs linked from the footer (path to files in <code>public/legal/</code>)	<code>Footer.astro</code>
<code>moduli</code>	<code>src/content/moduli/dati-stagionali.md</code>	Singleton	Season-specific data (fees, deadlines, week ranges) for the two enrollment forms	<code>SummerCampForm.astro</code> , <code>IscrizioneScuolaForm.astro</code>

8.2 THE `PAGINE` COLLECTION IN DEPTH

This is the largest and most important schema in the project. Every route on the site has exactly one corresponding Markdown file in `src/content/pagine/` (e.g. `tennis.md` feeds both `/tennis` and `/en/tennis`). The schema is a single flat Zod object containing:

- **Common fields**, present on every page: `title` , `description` (SEO), `hero_eyebrow` , `hero_titolo` , `hero_titolo_accent` , `hero_sottotitolo` , `hero_immagine` , `hero_immagine_alt` — plus the `_en` twin of each.

- **Page-specific body fields**, prefixed by page name (e.g. `storia_*`, `camps_*`, `scuola_*`, `pt_*`, `prep_*`, `tennis_*`, `padel_*`, `agonistica_*`, `torneo_*`, `partners_lista`) — all optional, since a given `.md` file only populates the block(s) relevant to its own page.
- **A shared `faq` array** usable by any "activity" page, rendered by `FaqAccordion.astro` and simultaneously emitted as JSON-LD `FAQPage` structured data (see §14.3). FAQ answers may contain raw HTML (`<strong>` , `<a href>`) — this is intentional, both the visual accordion and Google's FAQPage schema accept it.

See [Appendix C](#) for the condensed field reference per page. For the authoritative, complete list always check `src/content/config.ts` directly — it is heavily commented in Italian, explaining which fields belong to which page and why.

DESIGN RATIONALE: ONE SCHEMA, MANY OPTIONAL BLOCKS

Rather than one Content Collection per page type, the project uses a single `pagine` collection with an ever-growing flat schema. This keeps the TinaCMS sidebar to a single "Pagine" entry (with one template per page — see §9), at the cost of a very large schema file.

When adding a new page with a structured body (not just hero+SEO), follow the existing pattern: add a new set of optional, page-prefixed fields to the schema, mirrored in a new Tina template.

9. TINACMS — HEADLESS CMS

TinaCMS provides a visual, git-backed editing UI for non-technical content owners. It is **fully configured in the codebase** but requires TinaCloud credentials to be connected before the visual editor becomes usable in production. The site builds and functions correctly with or without those credentials — Tina is purely an editing convenience layer on top of the same Markdown files Astro reads directly.

9.1 CONFIGURATION FILE — `TINA/CONFIG.TS`

- Declares one Tina **collection** per content-collection folder (`eventi` , `servizi` , `planning` , `membership` , `info` , `legal` , `moduli` , `helpdesk` , `news`), each with an explicit field list (type, label, validation, UI widget) matching the Zod schema in `src/content/config.ts` .
- The `pagine` collection is declared with one **Tina template per page** (e.g. `generica` , `storia` , `summer_camps` , ...), so editors see one page-specific form per file instead of one giant form with hundreds of mostly-irrelevant fields.
- Fields needing translation carry a 🇬🇧 emoji prefix in their Tina label (e.g. "🇬🇧 Titolo (inglese)") so editors can tell at a glance which fields are the English counterpart.
- `media.tina.mediaRoot = ''` and `publicFolder = 'public'` — uploaded images are written directly into `public/` at the repository root (matching where the rest of the site's images already live — see §16).

THE TWO SCHEMAS ARE NOT AUTO-SYNCHRONIZED

`src/content/config.ts` (the Zod/Astro schema, which the build depends on) and `tina/config.ts` (the Tina UI schema, which only affects the editor) are two independent files that must be kept in sync **by hand**. If you add a field to one, you must add the equivalent field to the other, or the CMS UI and the actual page output will drift apart. Always update both together and double-check field names match exactly.

9.2 ACTIVATING THE VISUAL EDITOR

1. Create a project at app.tina.io and connect it to the GitHub repository.
2. Retrieve the **Client ID** and a **read/write token** from the TinaCloud dashboard.
3. In Vercel → Project Settings → Environment Variables, add `TINA_CLIENT_ID` and `TINA_TOKEN` .
4. Redeploy. The build command is already `npm run tina:build` (see §4.3), which generates the admin bundle into `public/admin` (per `build.outputFolder: 'admin'`).
5. The visual editor becomes reachable at `/admin` on the deployed site, authenticated through TinaCloud.

For local editing with Tina active, run `npm run tina:dev` with the same two environment variables exported in your shell (or in a local `.env` file, which Astro/Vite will load automatically — none is currently committed to the repo, by design, since it would contain a write-capable token).

9.3 BRANCH AWARENESS

`tina/config.ts` resolves the working Git branch dynamically: `process.env.VERCEL_GIT_COMMIT_REF || process.env.GITHUB_BRANCH || process.env.HEAD || 'main'` . On Vercel this is populated automatically per-deployment (including preview deployments for pull requests), so editors on a preview deployment edit that branch's content rather than always writing to `main` .

10. GLOBAL LAYOUT & CHROME

10.1 BASELAYOUT.ASTRO

Every page renders through this layout. It owns:

- **Document** `<head>` : charset/viewport/theme-color, favicons, title/description, canonical URL, `hreflang` alternates (it/en/x-default), Open Graph + Twitter Card tags, the site-wide `TennisComplex` JSON-LD block, and non-blocking Google Fonts loading (preload + `media="print"` onload swap trick, with a `<noscript>` fallback).
- **Consent & tracking bootstrap** (must stay first in `<head>` , see §13.4): Google Consent Mode v2 default state, the CookieYes banner script, Google Tag Manager, and a hand-written UTM-capture / persistent-visitor-id / "visit beacon" script bridged to CookieYes consent events.
- **Page chrome**: `Navbar` , `LeadFormInline` (unless `hideLeadFormInline`), `InfoPratiche` , `AffiliazioniSlider` , `Footer` , and the always-present `LeadModal` — all skippable via a `hideChrome` prop for embedded/in-app views (see §20.3).
- **Scroll-reveal animation system**: a single `IntersectionObserver` -driven script that auto-tags dozens of selectors across every page with the appropriate `data-anim` animation type (see §15.3) and a "back to top" button.

Props

PROP	TYPE	PURPOSE
<code>title</code>	string (required)	Page <code><title></code> and OG/Twitter title
<code>description</code>	string?	Meta description; falls back to a default tagline
<code>image</code> / <code>imageAlt</code>	string?	Social preview image (see §14.2 for the AVIF → JPG convention)
<code>lang</code>	<code>'it'</code> <code>'en'</code>	Defaults to auto-detection from the URL if omitted
<code>hideLeadFormInline</code>	boolean	Hides the inline contact form block at the bottom of the page
<code>hideChrome</code>	boolean	Hides navbar/footer/info/partners entirely (in-app embed pages)
<code>leadFormStandalone</code> , <code>leadFormTitle</code> , <code>leadFormEyebrow</code>	—	Customize the inline lead form when it is the primary content of a page (e.g. a dedicated "contact us" page)

10.2 NAVBAR.ASTRO

Fixed header with:

- Two **mega-menus** ("Adulti"/"Adults" and "Junior") driven by small in-file data arrays (label, description, href, feature image), with a CSS-only image-swap-on-hover effect using `:has()` .
- A **language switcher** dropdown (flag icons, uses `getAlternatePath` / `getLocalePath` to compute the equivalent URL in the other language).
- An **"Accedi" (Sign in) modal** — pure link hub, no authentication happens on this site: links out to the PerfectGym member portal (web login, class booking) and Playtomic (court booking), plus an app-store deep link. See §13.1.
- A responsive off-canvas mobile menu, and a "Contattaci" CTA that opens the shared lead modal (`data-open-modal="modal-contatti"` , wired in `LeadModal.astro`).

10.3 OTHER CHROME COMPONENTS

COMPONENT	ROLE
<code>Footer.astro</code>	Sitemap-style link columns, contact/social block, legal document links pulled from the <code>legal</code> collection, company registration details.

<code>InfoPratiche.astro</code>	"Visit us" block: address, hours, phone, embedded Google Map — sourced from the <code>info</code> collection. Deliberately excluded from scroll animations (see inline comment in <code>BaseLayout.astro</code>): animating a wrapper around a cross-origin Google Maps iframe causes a Chromium compositing bug where the map fails to render.
<code>AffiliazioniSlider.astro</code>	Logo slider for affiliated sports bodies, reading logo files from <code>public/logos/partners/</code> with graceful <code>.img-placeholder</code> fallback if a logo file is missing.
<code>LeadModal.astro</code> / <code>LeadFormInline.astro</code>	The two presentations (modal vs. inline-at-bottom-of-page) of the shared multi-step lead form — see §12.

11. COMPONENT REFERENCE

All components live flat in `src/components/`, one `.astro` file per component, PascalCase naming (project rule). Most accept an optional `lang` prop and render bilingual copy either from the i18n dictionaries or from an inline `lang === 'en' ? {...} : {...}` object for copy that doesn't belong in a content collection. See [Appendix B](#) for the full index with a one-line description of every component; the most structurally important ones are detailed inline through this document (Navbar §10.2, lead form components §12, FaqAccordion §14.3, MembershipSection §8, WeeklySchedule §8).

11.1 NOTABLE PATTERNS WORTH KNOWING BEFORE EDITING COMPONENTS

- **Homepage (`index.astro`) vs. activity pages:** the homepage composes many small, largely static/copy-driven components (`Hero`, `Manifesto`, `DiventaSocio`, `AttivitaSplit`, `Reassurance`, `TorneoAvvenire`, `EventiHome`, `ConversionBlock`), while activity pages (`tennis.astro`, `padel.astro`, `preparazione-atletica.astro`, etc.) are built from `PageHero` + inline `<section>` markup reading directly from the `pagine` collection entry, plus shared blocks like `MembershipSection` and `FaqAccordion`.
- **Every activity page is duplicated verbatim** between `src/pages/<page>.astro` and `src/pages/en/<page>.astro` — same section structure and scoped `<style>` block, only the copy source (``_en`` fallback) and a few hardcoded strings differ. When changing layout or styling, remember to apply the same change to both files.
- **Enrollment forms are stateful, multi-step, vanilla-JS forms** embedded directly in `IscrizioneScuolaForm.astro` and `SummerCampForm.astro` (not the shared lead form) — they read season-specific data from the `moduli` collection and bank details from `src/data/bankDetails.ts`, and are designed for bank-transfer enrollment confirmation flows rather than online payment.
- `DisciplineCard.astro` / `DisciplineGrid.astro` / `LeadFormPlaceholder.astro` appear to be earlier-iteration components not wired into any current page — verify usage with a repo-wide search before relying on or deleting them.

12. LEAD CAPTURE & FORMS SYSTEM

This is the most complex piece of client-side logic in the codebase and worth understanding in full before touching it.

12.1 SHARED ENGINE — `SRC/LIB/LEADFORM.CLIENT.JS`

Both `LeadModal.astro` (the "Contattaci" popup opened from any page) and `LeadFormInline.astro` (the same form embedded at the bottom of every page) render an identical multi-step form body (`LeadFormBody.astro`) and initialize it by calling a single shared function, `initLeadForm(root, { prefix, onReset })`, exported from `leadForm.client.js`. This consolidation replaced an earlier design where the same logic was duplicated in both components (a bug fix — timezone handling on the callback/visit date — once had to be applied twice; it is now applied once).

Flow

- Step 1 — email + activities of interest.** On "next", the form POSTs to the `tca-verifica-iscritto` n8n webhook with the email, selected activities, page/CTA context, UTM data and visitor id. The webhook responds with a status: `iscritto` (existing member), `esiste` (known contact, not a member) or `nuovo` (brand new), each optionally suffixed with `_adulto` / `_bambino` to indicate which audience group (adult vs. junior) the contact belongs to — this suffix is authoritative and can override the client-side guess based on selected activities.
- Step 2 (new contacts only) — identity.** First name, last name, mobile number (with country-code selector), privacy consent (required) and marketing consent (optional).
- Step 3 (adults only, existing/known contacts) — choose an action:** request a callback, book an in-person visit, or just send a free-text message. New junior contacts and already-registered members skip straight to the message step.
- Step 4 — the chosen action's detail screen:**
 - Callback:* an inline calendar (next 7 days) + time-slot picker (20-minute increments, business hours, weekday/weekend aware) + reason text.
 - Visit:* same idea, 14-day window, 30-minute slots, split morning/afternoon business hours.
 - Message:* a free-text box only.

On submit, each path POSTs the full accumulated state to the `tca-form-compilato` n8n webhook and fires a `generate_lead` GTM event (see §13.4) before showing a confirmation screen summarizing what was submitted. From there, the n8n workflow forwards the lead into PerfectGym, by email and into the operational CRM at `crm.tcambrosiano.com` — see §13.3 for the full downstream flow.

Calendar/slot generation rules

- All "today" availability checks use **Europe/Rome time** explicitly (via `Intl.DateTimeFormat` with an explicit `timeZone`), independent of the visitor's own browser timezone — this avoids a class of bugs where a visitor abroad would see (or not see) today's slots incorrectly.
- Same-day slots require at least a **2-hour lead time** from the current Milan time.
- The date sent to the backend is a plain `YYYY-MM-DD` string in local time, not `toISOString()` — `JSON.stringify(Date)` would convert to UTC and could silently shift a local midnight back a day for whoever consumes the payload without reconverting.

12.2 UTM & VISITOR-ATTRIBUTION

Handled by an inline script in `BaseLayout.astro` (not the lead form itself), exposed globally so any form/component can use it:

- `window.tcaGetUtm()` — returns the first-touch UTM/click-id parameters captured this session (from `sessionStorage` under `tca_utm`), falling back to whatever is in the current URL if nothing was stored yet.
- `window.tcaGetVid()` — a persistent random visitor id stored in `localStorage` (`tca_vid`), generated once per browser.
- `window.tcaTrack(eventName, data)` — pushes an event onto `window.dataLayer` for GTM.
- A "visit beacon" POSTs page/referrer/UTM/visitor-id to the `ping-tca` n8n webhook via `navigator.sendBeacon` (fetch fallback) — but only once analytics consent has been granted (see §13.4).

- UTM/click-id parameters are also **propagated across internal navigation** (rewriting same-origin link hrefs on click) so a visitor who lands from a campaign keeps that attribution as they browse, without needing cookies.

12.3 OTHER FORMS

COMPONENT	PURPOSE	DATA SOURCE
<code>IscrizioneScuolaForm.astro</code>	Tennis School enrollment (children), including size/level selection, prep-course time slot, and bank-transfer payment instructions	<code>moduli</code> collection (deadlines, deposit amount, age cohorts) + <code>src/data/bankDetails.ts</code>
<code>SummerCampForm.astro</code>	Summer Camp week-by-week enrollment, with weeks auto-hidden once their end date has passed	<code>moduli</code> collection (weekly fee table, week date ranges) + <code>bankDetails.ts</code>
<code>InviteFriendModal.astro</code>	Member-referral form ("Invite a Friend"), reachable from Club Life	Static copy, phone country-code list from <code>src/data/prefissi-telefonici.ts</code>
<code>ClubLifeHelpForm.astro</code>	Support-request entry point on the Club Life page, opens the shared lead modal	—

13. THIRD-PARTY INTEGRATIONS

13.1 PERFECTGYM

Membership management platform, used in two distinct ways:

- **Outbound links** — the site links out (no API integration, no embedded iframe) to the member portal login, online registration and the class-booking calendar.
- **Inbound lead destination** — every lead submitted through the site's lead-capture form (§12) is also delivered *into* PerfectGym as a new lead/contact record, so front-desk and sales staff work from PerfectGym itself rather than from a separate inbox. This delivery is not code in this repository: it happens inside the n8n automation the site's webhooks feed into (§13.3) — PerfectGym's side of the integration lives entirely in n8n/PerfectGym configuration, outside this codebase.

All outbound PerfectGym URLs are centralized in `src/data/links.ts` (the `perfectGym` export: `login`, `register`, `classBooking(lang)`) and imported wherever needed (Navbar's "Accedi" modal, homepage CTAs, `MembershipSection.astro`, `WeeklySchedule.astro`). A URL change on PerfectGym's side now only requires editing that one file.

13.2 PLAYTOMIC

Court-booking platform, linked from the "Accedi" modal and directly from the Tennis/Padel activity pages. Also centralized in `src/data/links.ts` (the `playtomic` export: `club`, `tennisCourt`, `padelCourt`).

LINK CENTRALIZATION (§13.1–13.2)

Prior to this being centralized, every PerfectGym/Playtomic/app-store URL was hardcoded inline, once per occurrence, across Navbar, Hero, DiventaSocio, ConversionBlock, MembershipSection, WeeklySchedule and the Tennis/Padel pages (IT+EN) — a URL change meant a manual find-and-replace across the codebase, with real risk of missing an occurrence. All of these now live in `src/data/links.ts` (`perfectGym`, `playtomic`, `memberApp` exports) and every component/page imports from there. Centralizing also surfaced and fixed a latent bug: the class-booking link in `WeeklySchedule.astro` was hardcoded to `?lang=it` even when rendered on the English site (`/en/preparazione-atletica`), since the component ignored its own `lang` prop for that one URL. When adding a new third-party link anywhere in the site, add it to `src/data/links.ts` rather than inlining it in a component.

13.3 N8N (MARKETING AUTOMATION BACKEND) & DOWNSTREAM SYSTEMS

All lead/visit data flows to a self-hosted n8n instance at `automazione.n8ndevelop.it` via three webhooks:

WEBHOOK	CALLED FROM	PURPOSE
<code>/webhook/ping-tca</code>	<code>BaseLayout.astro</code> inline script	Anonymous visit beacon (page, referrer, UTM, visitor id) — analytics-consent gated
<code>/webhook/tca-verifica-iscritto</code>	<code>leadForm.client.js</code> , step 1	Looks up whether the submitted email belongs to an existing member/known contact and which audience group they belong to
<code>/webhook/tca-form-compilato</code>	<code>leadForm.client.js</code> , final step of every flow	Delivers the completed lead (contact details, activities, chosen action, date/time or message, UTM, visitor id) into the marketing-automation pipeline

This is the site's **only backend dependency** for form functionality — if these endpoints change domain or path, update the constants at the top of `leadForm.client.js` and the inline script in `BaseLayout.astro`.

Where lead data goes after n8n

The website's own responsibility ends at the `tca-form-compilato` webhook call — from there, the n8n workflow (not part of this repository; configured directly in n8n) fans the submission out to three destinations:

1. **PerfectGym** — creates/updates a lead record in the club's membership-management platform (§13.1), so staff work the lead from the same system used for members.

2. **Email** — sends a notification email (recipients and template are configured in n8n, not in this codebase).
3. **Operational CRM reporting** — forwards the contact into a lightweight internal CRM at `crm.tcambrosiano.com`, used for operational reporting/tracking of contacts rather than as the club's primary CRM.

THIS FAN-OUT IS NOT VISIBLE IN THIS REPOSITORY

The website only ever calls the three n8n webhook URLs above — it has no direct integration with PerfectGym's lead API, no SMTP/email code, and no connection to `crm.tcambrosiano.com`. The PerfectGym/email/CRM fan-out is entirely n8n-side configuration. If a future agency needs to change *where* submitted leads end up, that change happens in the n8n workflow (and/or in `crm.tcambrosiano.com`'s own configuration), not in this codebase — this document records the business flow for completeness, but the n8n workflow itself is the source of truth for its exact logic and is outside this repository's scope.

13.4 GOOGLE TAG MANAGER, CONSENT MODE V2 & COOKIEYES

Consent handling is implemented carefully and in a specific script order that **must not be reordered**:

1. **Consent Mode v2 default state** is set first — everything denied except strictly-necessary storage, with a 500ms `wait_for_update` grace window for CookieYes to restore a returning visitor's prior choice before any tag actually fires.
2. **CookieYes banner script** loads next (client ID embedded in the script URL) — it is the source of truth for the visitor's consent choice.
3. **Google Tag Manager** (container `GTM-WRDQ2RTG`) loads unconditionally (not gated behind a script-type trick), but starts in the denied Consent Mode state and stays cookie-less/anonymous-ping-only until consent is granted — this is Google's recommended approach and the only one compatible with conversion modeling and (in the EEA) remarketing.
4. A custom bridge listens for CookieYes's `cookieyes_banner_load` (returning visitor, already consented previously) and `cookieyes_consent_update` (visitor accepts/changes now) events, and calls `gtag('consent', 'update', ...)` accordingly — CookieYes's own native Consent Mode integration must stay **disabled** in the CookieYes dashboard, since enabling it would create a second signal source firing the same updates.

The visit beacon and the persistent visitor id (§12.2) are only activated once analytics consent is confirmed via this bridge; UTM capture into storage is likewise consent-gated, but simply *reading* UTM parameters from the current URL (for immediate use, e.g. attributing a lead submitted on the landing page itself) is treated as URL inspection rather than storage and is not consent-gated — this legal reasoning is documented inline in the layout's comments and should be preserved if this code is ever touched.

13.5 MEMBER APP

A deep link to the club's member app is provided (`https://onelink.to/7vfz2`, a OneLink/AppsFlyer smart link). Pages opened *inside* that app's webview pass `?app=1` in the URL; a tiny inline script in `BaseLayout.astro` detects this before first paint and adds an `is-app-embed` class to `<html>`, which `global.css` uses to hide the navbar, footer, info block and back-to-top button (see §20.3) — purely presentational, no server-side logic involved.

14. SEO IMPLEMENTATION

14.1 PER-PAGE METADATA

Every page supplies `title / description` (from the `pagine` collection, with `_en` fallback on English pages) to `BaseLayout`, which renders: the `<title>` and meta description, a canonical URL, and reciprocal `hreflang` alternates (`it`, `en`, and `x-default` pointing at the Italian version).

14.2 SOCIAL PREVIEWS

Open Graph and Twitter Card tags use the page's hero image. Because Facebook/LinkedIn/WhatsApp link-preview crawlers do not reliably read AVIF, the layout automatically swaps any `.avif` hero image path to its `.jpg` counterpart (same filename, a 1200×630 twin generated alongside the AVIF) for the `og:image / twitter:image` tags specifically — **every hero image must be exported in both formats** for this to work; see §16.2.

14.3 STRUCTURED DATA (JSON-LD)

SCHEMA TYPE	WHERE	SCOPE
<code>TennisComplex</code>	<code>BaseLayout.astro</code>	Site-wide: name, address, phone, email, opening hours, social profile links. This is the most specific schema.org type available for a tennis club.
<code>FAQPage</code>	<code>FaqAccordion.astro</code>	Per-page, only rendered when the page's <code>faq</code> content-collection field is populated. Feeds both the visual accordion and search-engine/LLM-readable rich results.

14.4 SITEMAP & ROBOTS

`@astrojs/sitemap` generates an i18n-aware sitemap (`sitemap-index.xml`) automatically from the built routes, configured with the same `it / en` locale map as `astro.config.mjs`. `public/robots.txt` allows all crawling and points to the sitemap on the production domain.

15. STYLING SYSTEM

15.1 DESIGN TOKENS

All defined as CSS custom properties on `:root` in `src/styles/global.css` (mirrored for quick reference in `CLAUDE.md`):

TOKEN	VALUE	USE
<code>--color-bg</code>	<code>#FFFFFF</code>	Page background
<code>--color-text</code>	<code>#1A1A1A</code>	Primary text
<code>--color-text-light</code>	<code>#555555</code>	Secondary/body text
<code>--color-accent</code>	<code>#8B1A1A</code>	Brand red — CTAs, links, accents
<code>--color-accent-dark</code>	<code>#6B1414</code>	Hover state of accent
<code>--color-border</code>	<code>#EBEBEB</code>	Borders, dividers
<code>--color-off-white</code>	<code>#F5F3F0</code>	Alternate section backgrounds
<code>--color-overlay</code>	<code>rgba(0,0,0,0.55)</code>	Hero image overlays
<code>--max-width</code>	<code>1500px</code>	Max content width
<code>--pad-x</code>	<code>clamp(20px,5vw,72px)</code>	Fluid horizontal padding
<code>--header-h</code>	<code>60px (52px <768px)</code>	Fixed navbar height, referenced by scroll-margin/offsets throughout
<code>--font-display</code> / alias <code>--font-serif</code>	<code>"Barlow Condensed"</code>	Headings — always uppercase, weight 800
<code>--font-body</code> / alias <code>--font-sans</code>	<code>"Barlow"</code>	Body copy — weights 300/400

15.2 UTILITY CLASSES

- `.eyebrow` / `.eyebrow--light` / `.eyebrow--grey` — small uppercase red label above section titles, with light/grey variants for dark or muted backgrounds.
- `.btn`, `.btn-red`, `.btn-outline`, `.btn-outline--dark` — the only button styles in the system.
- `.img-placeholder` — a bordered grey box showing the intended filename (via `data-name` + CSS `content: attr(data-name)`), used wherever real photography is not yet available (project convention, see §20.1). *Search the codebase for this class before a launch to catch any remaining placeholders.*
- `.rich-body` — typography reset applied to any block rendering Tina rich-text or converted markdown (headings, lists, links, blockquotes), so long-form content matches the design system without extra per-component CSS.

15.3 SCROLL-REVEAL ANIMATION SYSTEM

A single script in `BaseLayout.astro` auto-registers most section headings, eyebrows, cards, stats and images across the entire site with a `data-anim` attribute and observes them with one shared `IntersectionObserver` (5% visibility threshold). CSS then animates based on the attribute's value:

VALUE	EFFECT	TYPICAL USE
<code>(default/ true)</code>	Slide-up + fade + blur-in	Generic cards/blocks
<code>reveal</code>	Slide-from-left with slight overshoot	Section <code><h2></code> / <code><h3></code>
<code>from-left</code>	Horizontal slide + fade	Eyebrows, paragraphs, body text
<code>scale</code>	Scale-in + rise	Photos, stat numbers
<code>pop</code>	Bounce/overshoot	Buttons/CTAs

fade	Opacity only	Elements on dark panels where a blur/transform looks wrong
------	--------------	--

Elements can pre-set their own `data-anim / data-anim-delay` in markup to override the auto-registration list. Hero sections are explicitly excluded (always visible on load, never animated). The whole system respects `prefers-reduced-motion` (animations are fully disabled, elements shown immediately) and has a fallback that reveals everything if `IntersectionObserver` is unsupported.

One deliberate exception: `.info__map` (the embedded Google Map) is never animated — applying transform/filter to a wrapper around a cross-origin iframe triggers a Chromium compositing bug that leaves the map blank.

15.4 RESPONSIVE & ACCESSIBILITY SAFEGUARDS

- A global mobile rule enforces an **11px minimum font size** across a broad selector list (paragraphs, labels, footer text, BEM `__label/ __meta/ __sub/ __desc/ __link/ __footer/ __value` suffixes) below 768px, as a safety net against `clamp()` values that might otherwise get too small.
- A print stylesheet hides all chrome (navbar, footer, info, partners slider, back-to-top, any `<dialog>`) so printed pages show content only.
- `html, body { overflow-x: clip }` prevents horizontal scroll from off-screen slide-in animations without breaking `position: sticky` elements (used by the membership table and events list) — deliberately `clip` rather than `hidden`.

16. ASSETS & MEDIA CONVENTIONS

16.1 WHERE IMAGES ACTUALLY LIVE

`CLAUDE.md` documents a target structure under `src/assets/images/{hero, disciplines, torneo, club}/` and `src/assets/logos/partners/` for curated photography. **In the current codebase these folders are empty placeholders** (each contains only a `.gitkeep` file) — every image actually referenced by a page or component today lives **flat inside `public/`** (over 400 files at the time of writing) and is addressed via the `publicUrl()` helper (§7.1) so it resolves correctly under both the Vercel root deployment and the GitHub Pages subpath.

When adding new photography, follow whichever pattern the surrounding page already uses; if migrating the project toward the documented `src/assets/` structure, note that Astro's asset pipeline (`import`-based, optimized) behaves differently from static files in `public/` (served as-is, no processing) and existing image markup would need to change accordingly — this is a larger, deliberate refactor, not a drop-in move.

16.2 FORMAT CONVENTION: AVIF + JPG TWINS

Hero and key content images are exported as both `.avif` (used on-page, for its size/quality advantage) and a same-named `.jpg` twin sized 1200×630 — the format/dimensions recommended for Open Graph — because social-preview crawlers (Facebook, LinkedIn, WhatsApp) do not reliably decode AVIF. See §14.2 for how `BaseLayout` automatically substitutes the `.jpg` for social tags. **Any new hero image must ship both files**, or social share previews for that page will silently fall back to the sitewide default image.

16.3 MISSING PHOTOGRAPHY

Per `CLAUDE.md`: never insert placeholder/lorem-ipsum copy or a fake stock photo. Use the `.img-placeholder` utility (§15.2) — a bordered box that displays the intended filename via `data-name` — until the real asset is supplied.

16.4 OTHER STATIC ASSETS

- `public/legal/` — the two PDFs (Codice di Condotta, Nomina Responsabile Safeguarding) linked from the footer, referenced by the `legal` content collection. Replacing a document is just uploading a new PDF at the same path (or a new path + updating the `legal` entry).
- `public/email/` — images referenced by external transactional email templates (e.g. built in n8n), not used by the website itself but kept in the repo for asset continuity.
- `public/logos/` — site logo variants (extended wordmark for desktop, compact mark for mobile) and the `partners/` subfolder for affiliate logos.

17. CONTENT EDITING CONVENTIONS

These conventions matter both for whoever edits content through TinaCMS and for developers hand-editing Markdown files directly.

17.1 BOLD TEXT IN PLAIN-STRING FIELDS

Any field documented as accepting "**bold with **double asterisks****" (most `_en` long-text fields, plus a few Italian lead-paragraph fields rendered with a small inline `mdBold()` helper directly in the page component) uses simple `**text**` markdown-style bold — nothing else is parsed there (no italics, no links) unless the field explicitly says otherwise.

17.2 LONG-FORM LEGAL TEXT — `SRC/LIB/MINIMARKDOWN.TS`

Two fields (`regolamento_body`, `privacy_body` and their `_en` counterparts) hold long-form legal text authored as plain strings but supporting a minimal, hand-rolled markdown subset: `##` / `###` headings, `-` bullet lists, blank-line paragraphs and `**bold**` — deliberately not a full markdown library, since these are the only constructs these particular documents need.

17.3 FAQ FIELDS ALLOW RAW HTML

Unlike other plain-text fields, FAQ answers (`faq[].risposta` / `risposta_en`) may contain literal `<strong>` and `<a href="/path">` tags, rendered as-is. This is intentional (documented directly in `tina/config.ts`'s field label) and matches what Google's FAQPage rich-result schema itself accepts in its `text` property.

17.4 RICH-TEXT (ITALIAN) VS. PLAIN-TEXT (`_EN`)

See §7.3 — this asymmetry is intentional and should not be “fixed” by trying to give English rich-text without first restructuring the relevant Tina collection/schema.

18. DEPLOYMENT & CI/CD

18.1 VERCEL (PRODUCTION)

Configured via `vercel.json`: `framework: "astro"`, `buildCommand: "npm run tina:build"`, `outputDirectory: "dist"`. Deploys automatically on push to `main` via Vercel's native Git integration (no explicit GitHub Actions workflow needed for this environment — Vercel's own webhook/build system handles it once the repository is connected in the Vercel dashboard).

18.2 GITHUB PAGES (PREVIEW)

`.github/workflows/deploy.yml` — triggered on push to `main` (and manually via `workflow_dispatch`). Builds with `GITHUB_PAGES=true`, which flips `astro.config.mjs` to use the GitHub Pages site URL and the `/WebSite-TCA` base path, then publishes `dist/` via the official `actions/deploy-pages` action.

18.3 CONTINUOUS INTEGRATION — `.GITHUB/WORKFLOWS/CI.YML`

Runs on every pull request into `main` and every push to `main`:

1. `npm run check:redirects` — fast static check (self-loop detection, does not need a build)
2. `npm run check` — `astro check` (TypeScript/template type errors)
3. `npm run build` — full production build
4. `npm run check:redirects` again — this second pass compares `vercel.json` redirect rules against the *actual built routes* in `dist/`, which is only possible after the build exists

18.4 `VERCEL.JSON` REDIRECTS & THE VALIDATOR SCRIPT

`vercel.json` carries roughly 60 redirect rules cleaning up legacy URLs from a previous site structure (old `/it/milan/club-ambrosiano/...` paths, old English routes, retired sections). `scripts/check-vercel-redirects.mjs` exists because of two real production incidents and guards against their root causes on every CI run:

- **Self-loop bug:** a rule like `"/agonistica/:path*" → "/agonistica"` also matches the bare path with *zero* segments (`:path*` means "zero or more"), redirecting the page to itself — an infinite loop, effectively taking the page offline. This previously broke `/agonistica`, `/tennis`, and the Italian `/news` route. Fix pattern: use `:path+` ("one or more") whenever the bare prefix must remain a real, reachable page.
- **Shadowing real pages:** a wildcard rule meant to clean up a legacy prefix can accidentally match *current, real* pages if they share that prefix — this previously broke the entire English section of the site (a rule `"/en/:path+" → "/en"`) and the English news pages (`"/en/news/:path*" → "/en/club-life"`). The script's second pass catches this by diffing wildcard rules against the routes Astro actually generated.

WHEN ADDING A NEW REDIRECT RULE

Always run `npm run build && npm run check:redirects` locally before pushing — the build-aware shadowing check needs `dist/` to exist, and CI will fail the same way it would locally, just later.

19. ENVIRONMENT VARIABLES & SECRETS

VARIABLE	WHERE SET	REQUIRED?	PURPOSE
TINA_CLIENT_ID	Vercel project env vars (and local shell/.env for tina:dev)	Only to enable the live TinaCMS visual editor	TinaCloud project identifier (see §9.2)
TINA_TOKEN	Same as above	Only to enable the live TinaCMS visual editor	TinaCloud read/write token — treat as a secret, never commit it
GITHUB_PAGES	Set inline by .github/workflows/deploy.yml	Only for the GitHub Pages build	Switches astro.config.mjs's site / base to the GitHub Pages subpath configuration
VERCEL_GIT_COMMIT_REF / GITHUB_BRANCH / HEAD	Auto-populated by the hosting platform	No manual action needed	Tells tina/config.ts which Git branch the current build/edit session belongs to

There is no .env file committed to the repository (by design — it would need to contain the write-capable Tina token). No API keys are required for the site to build and run without CMS editing; the n8n webhook URLs, GTM container id and all third-party links are plain constants in the source, not secrets.

20. KNOWN CONVENTIONS, CONSTRAINTS & GOTCHAS

20.1 HOUSE RULES (FROM `CLAUDE.MD`)

- No CSS framework, no UI component library — plain CSS with custom properties only.
- Never write placeholder/lorem-ipsum copy; missing photography uses `.img-placeholder`, never a stock photo.
- One component per file, PascalCase filenames.
- The service currently called "Preparazione Atletica" in Italian must **never** be translated or labeled as "fitness" anywhere, including in SEO/meta fields — it is a deliberate brand-naming decision, not an oversight.
- Development happens on `feature/<name>` branches; `main` is production. Per the project's own workflow notes, commits/pushes are handled manually by the site owner rather than automatically by an assistant — confirm this expectation with the current maintainers before changing that process.

20.2 GITHUB PAGES BASE-PATH HANDLING

Every internal link, and every reference to a file under `public/`, must be built with `getLocalePath()` or `publicUrl()` (§7.1) rather than a hardcoded absolute path. Since the GitHub Pages deployment serves the site from `/WebSite-TCA/`, a hardcoded `/tennis` or `/some-image.avif` will 404 there even though it works fine on Vercel's root-served production domain.

20.3 IN-APP EMBED MODE (`?APP=1`)

Pages can be opened inside the member app's webview with `?app=1` in the query string. This is detected client-side before first paint (an inline script tags `<html>` with `is-app-embed`) and is purely a CSS switch (hides navbar/footer/info/back-to-top) — there is no separate server-rendered "app" variant of any page, and no build-time branching. The same static HTML is served either way.

20.4 REDIRECT RULE QUANTIFIERS

See §18.4 — `:param*` vs. `:param+` in `vercel.json` is a recurring, easy-to-miss source of production outages for entire site sections. Always run the redirect checker (§4.3, §18.4) after touching `vercel.json`.

20.5 TWO SCHEMAS, ONE SOURCE OF TRUTH EACH

`src/content/config.ts` (build-time validation) and `tina/config.ts` (editor UI) must be updated together whenever a content field is added, renamed or removed (§9.1) — there is no tooling that keeps them in sync automatically.

20.6 COMPONENTS THAT MAY BE DEAD CODE

`DisciplineCard.astro`, `DisciplineGrid.astro` and `LeadFormPlaceholder.astro` were not found wired into any current page during this review. Confirm with a repository-wide usage search before relying on them for new work, or before removing them.

21. MAINTENANCE PLAYBOOK

21.1 ADD A BRAND-NEW PAGE

1. Add a new entry to the `pagine` collection: create `src/content/pagine/<slug>.md` with at least `title` / `description` (and any hero/body fields you plan to use — extend the Zod schema in `src/content/config.ts` first if the page needs new structured body fields).
2. Add the matching Tina template in `tina/config.ts` (copy the closest existing template, e.g. `generica` for a simple hero+SEO page) so editors can manage it visually.
3. Create `src/pages/<slug>.astro` : import `BaseLayout` , fetch the entry with `getEntry("pagine", "<slug>")` , render the page body.
4. Duplicate it as `src/pages/en/<slug>.astro` : set `lang = 'en'` , use the same content entry but read every field as `d.field_en ?? d.field` , and localize any hardcoded inline strings.
5. Link the new page from wherever it should be discoverable (Navbar, footer, related pages) using `getLocalePath` .
6. Run `npm run check && npm run build` before committing.

21.2 ADD A NEW EVENT / NEWS ARTICLE / HELP-DESK GUIDE / SERVICE CARD

1. Add a new Markdown file to the relevant collection folder (`src/content/eventi|news|helpdesk|servizi/`), following the schema in `src/content/config.ts` for required fields.
2. Fill in the Italian body (rich-text/markdown as appropriate) and, when ready, the `_en` / `corpo_en` fields.
3. These collections render automatically from `getCollection()` calls (with a `pubblicato` filter for events/news) — no code change needed for a routine new entry, only for a new field.
4. This can equally be done through the TinaCMS editor once activated (§9.2), without touching Markdown directly.

21.3 UPDATE SEASONAL PRICING / DATES (SUMMER CAMP, TENNIS SCHOOL)

1. Edit `src/content/moduli/dati-stagionali.md` (or the equivalent TinaCMS "Dati Stagionali Form" entry) — fees, deposit amounts, deadlines, week date ranges.
2. No code changes are required; field labels/copy live in the form components, only the season-specific values live in this file.

21.4 ADD A REDIRECT

1. Add the rule to `vercel.json`'s `redirects` array.
2. If the rule uses a wildcard, prefer `:path+` unless you specifically intend the bare prefix to also redirect.
3. Run `npm run build && npm run check:redirects` locally and confirm it passes before pushing (see §18.4).

21.5 ACTIVATE/VERIFY TINACMS IN PRODUCTION

Follow §9.2 in full. Once live, verify by visiting `/admin` on the production domain and confirming login against TinaCloud succeeds.

21.6 REPLACE A LEGAL DOCUMENT

1. Add the new PDF under `public/legal/` .
2. Update the corresponding field in `src/content/legal/documenti.md` (or via Tina's "Documenti Legali" entry, which is an image/file-upload field pointed at this same collection) to reference the new filename.

21.7 UPDATE OR EXTEND TRANSLATIONS

- **Static UI copy** (buttons, nav, form labels): edit `src/i18n/it.ts` / `en.ts` directly — keep both objects structurally identical (same keys).
- **Editorial content** (page copy, events, news, FAQ...): fill in the corresponding `_en` field in the content collection entry (directly or via TinaCMS). Leaving it blank is safe — the English page will show the Italian copy for that field until translated (§7.2).

End of document. For anything not covered here, the most reliable next step is reading the relevant source file directly — the codebase is consistently and thoroughly commented (in Italian) explaining intent, not just mechanics, especially in `src/content/config.ts`, `tina/config.ts`, `BaseLayout.astro` and `leadForm.client.js`.

APPENDIX A — FULL ROUTE TABLE

ITALIAN ROUTE	ENGLISH ROUTE	PAGE PURPOSE
/	/en	Homepage
/tennis	/en/tennis	Adult Tennis
/padel	/en/padel	Padel & Pickleball
/preparazione-atletica	/en/preparazione-atletica	Athletic Training (gym + group classes)
/personal-trainer	/en/personal-trainer	Personal Training
/scuola-tennis	/en/scuola-tennis	Tennis School (children)
/agonistica	/en/agonistica	Competitive Tennis Academy
/summer-camps	/en/summer-camps	Summer Camps
/torneo-avvenire	/en/torneo-avvenire	Avvenire Tournament (youth tournament, est. 1965)
/storia	/en/storia	Club history
/eventi	/en/eventi	Events listing
/eventi/[slug]	/en/eventi/[slug]	Single event detail
/club-life	/en/club-life	Club Life hub (news, help desk, services, referral)
/news/[slug]	/en/news/[slug]	Single news article
/club-life/help-desk/[slug]	/en/club-life/help-desk/[slug]	Single help-desk guide
/partners	/en/partners	Partners listing
/planning	/en/planning	Weekly group-class schedule
/iscrizioni	/en/iscrizioni	Membership/enrollment overview
/iscrizione-scuola-tennis	/en/iscrizione-scuola-tennis	Tennis School enrollment form
/iscrizione-summer-camp	/en/iscrizione-summer-camp	Summer Camp enrollment form
/prenota-una-visita	/en/prenota-una-visita	Book-a-visit landing page
/contatti	/en/contatti	Contact page
/regolamento	/en/regolamento	Club rules
/privacy-policy	/en/privacy-policy	Privacy policy
/link	/en/link	Link-in-bio style page (social profiles)
/tennis-milano	—	SEO landing page (Italian only)
/404	—	Not-found page

This list reflects `src/pages/` at the time of writing; treat the source directory itself as authoritative for any page added since.

APPENDIX B — FULL COMPONENT INDEX

COMPONENT	ONE-LINE PURPOSE
AffiliazioniSlider.astro	Logo slider for affiliated sports bodies (public/logos/partners/), with placeholder fallback
AttivitaSplit.astro	Homepage "Adults / Junior" split navigation panel
ClubLifeAssistBtn.astro	Floating "request assistance" button on Club Life, opens the lead modal
ClubLifeHelpDesk.astro	Club Life help-desk section: category filters + searchable guide list, reads the <code>helpdesk</code> collection
ClubLifeHelpForm.astro	Club Life "need help?" call-to-action section
ClubLifeHeroMenu.astro	Club Life hero quick-links (Events/News/Help Desk/Services), includes InviteFriendModal
ClubLifeLangSwitch.astro	Small standalone language-switch link (used inside Club Life sub-pages)
ClubLifeNews.astro	Club Life "news from the club" section, reads the <code>news</code> collection
ClubLifeServizi.astro	Club Life "Services & Partners" card grid, reads the <code>servizi</code> collection
ConversionBlock.astro	Bottom-of-page "become a member / book a visit" dual-CTA block
DisciplineCard.astro / DisciplineGrid.astro	Generic discipline card grid — appears unused by current pages, verify before relying on it (§20.6)
DiventaSocio.astro	Homepage membership pitch with benefit list
EventiHome.astro	Homepage "upcoming events" list, reads the <code>eventi</code> collection
FaqAccordion.astro	FAQ accordion + emits JSON-LD FAQPage structured data (§14.3)
Footer.astro	Site footer: links, contact, legal documents, company registration details
Hero.astro	Homepage full-screen hero section
InfoPratiche.astro	"Visit us" block: address/hours/phone/map, reads the <code>info</code> collection
InviteFriendModal.astro	Member-referral ("Invite a Friend") form
IscrizioneScuolaForm.astro	Tennis School enrollment form
LeadFormBody.astro	Shared multi-step form markup used by both LeadModal and LeadFormInline
LeadFormInline.astro	Inline lead-capture form, rendered near the bottom of every page
LeadFormPlaceholder.astro	Earlier/simplified lead-form markup — appears unused by current pages (§20.6)
LeadModal.astro	Modal presentation of the shared lead form, opened via <code>data-open-modal="modal-contatti"</code>
Manifesto.astro	Homepage brand-story section
MembershipSection.astro	Silver/Gold/Platinum pricing table, reads the <code>membership</code> collection
Navbar.astro	Fixed header, mega-menus, language switcher, "Accedi" modal, mobile menu (§10.2)
PageHero.astro	Generic page-header component used by every non-homepage activity page
PhotoLightbox.astro	Shared full-screen image lightbox dialog
Reassurance.astro	"Why choose us" reasons section
SummerCampForm.astro	Summer Camp enrollment form
TorneoAvvenire.astro	Homepage Avvenire Tournament teaser section
WeeklySchedule.astro	Weekly group-class timetable, reads the <code>planning</code> collection

APPENDIX C — CONTENT COLLECTIONS FIELD REFERENCE

(CONDENSED)

This is a condensed map of the `pagine` collection's page-specific field prefixes. Every field also has hero/SEO common fields (§8.2) and, in almost every case, an `_en` twin. For exact field names/types, always check `src/content/config.ts` directly — it is the single source of truth and is kept meticulously commented.

PREFIX	PAGE	NOTABLE STRUCTURES
<code>storia_*</code>	Club history	Timeline array (year + text), photo gallery
<code>camps_*</code>	Summer Camps	Feature list, daily schedule, weekly technical program, packing list, pricing rows
<code>scuola_*</code>	Tennis School	Skill-level list (color-coded), enrollment step cards, tournament list, quick-info + pricing rows
<code>pt_*</code>	Personal Trainer	Stat counters, trainer profiles (photo, specializations, qualifications, languages), process steps
<code>prep_*</code>	Athletic Training	Gym slideshow + features, class categories (endurance/strength/balance) each with a nested course list
<code>tennis_*</code>	Adult Tennis	Court feature cards, membership-note text, course-level cards
<code>padel_*</code>	Padel	Course-level cards
<code>agonistica_*</code>	Competitive Academy	Method pillars, key-points list, enrollment steps
<code>torneo_*</code>	Avvenire Tournament	Quote block, history text, stat counters, alumni intro, gallery, "next edition" block
<code>faq</code>	Any activity page	Shared across pages; HTML allowed in answers (§17.3)
<code>partners_lista</code>	Partners	Name, description, logo, external link, per entry
<code>regolamento_body</code> / <code>privacy_body</code>	Club Rules / Privacy Policy	Long-form text, mini-markdown syntax (§17.2)

Other collections — key fields

COLLECTION	KEY FIELDS
<code>eventi</code>	<code>titolo</code> , <code>data</code> , <code>categoria</code> (Torneo/Clinic/Evento/Camp), <code>descrizione</code> , <code>luogo</code> , <code>iscrizioniHref</code> , <code>pubblicato</code> , rich-text body
<code>news</code>	<code>titolo</code> , <code>data</code> , <code>categoria</code> , <code>sintesi</code> , <code>immagine</code> / <code>immagine_alt</code> , <code>pubblicato</code> , optional CTA (<code>cta_label</code> / <code>cta_href</code>), rich-text body
<code>helpdesk</code>	<code>titolo</code> , <code>categoria</code> (enum, mirrored in <code>src/data/helpDesk.ts</code>), <code>sintesi</code> , <code>tags[]</code> , <code>aggiornato</code> (date), rich-text body
<code>servizi</code>	<code>titolo</code> , <code>ordine</code> (grid position), <code>icon</code> (enum of 8 built-in SVG icons), <code>desc</code> , <code>dettaglio</code> , optional <code>href</code>
<code>planning</code>	<code>lezioni[]</code> : day, time, category (Strength/Balance/Endurance), course name
<code>membership</code>	Titles/names for the three tiers, a <code>categorie[]</code> array of table sections, each with <code>righe[]</code> rows (feature name + per-tier cell type: included/partial/value/check)
<code>info</code>	<code>indirizzo</code> , <code>come_arrivare_en</code> , <code>orari_en</code> , <code>telefono</code> , <code>email</code>
<code>legal</code>	<code>codice_condotta</code> , <code>nomina_responsabile</code> — both PDF file paths
<code>moduli</code>	Summer Camp weekly fee schedule + week date ranges; Tennis School deadlines, deposit, trial-lesson periods, eligible birth-year ranges

APPENDIX D — THIRD-PARTY ENDPOINTS & EXTERNAL LINKS

Unless noted otherwise, every URL below is defined in `src/data/links.ts` (§13.1–13.2) or as a constant near its point of use (n8n webhooks in `leadForm.client.js` / `BaseLayout.astro`).

SERVICE	ENDPOINT / URL PATTERN	USED FOR
n8n — visit beacon	<code>https://automazione.n8ndevelop.it/webhook/ping-tca</code>	Anonymous page-visit tracking (consent-gated)
n8n — membership check	<code>https://automazione.n8ndevelop.it/webhook/tca-verifica-iscritto</code>	Lead-form step 1: checks if an email belongs to a known/existing member
n8n — lead submission	<code>https://automazione.n8ndevelop.it/webhook/tca-form-compilato</code>	Final submission of every lead-form flow (callback/visit/message); fans out server-side (in n8n, not in this repo) to PerfectGym, email and the CRM below — see §13.3
Google Tag Manager	Container ID <code>GTM-WRDQ2RTG</code>	Analytics/Ads tag orchestration, gated by Consent Mode v2
CookieYes	<code>https://cdn-cookieyes.com/client_data/a52d1f8cdcea8593d12025ba01e05026/script.js</code>	Cookie consent banner + consent-state source of truth
PerfectGym — portal login	<code>https://tcambrosiano.perfectgym.com/clientportal2/#/Login</code>	Member self-service portal (<code>perfectGym.login</code>)
PerfectGym — registration	<code>https://tcambrosiano.perfectgym.com/clientportal2/#/Registration</code>	Online membership sign-up (<code>perfectGym.register</code>)
PerfectGym — class booking	<code>https://tcambrosiano.perfectgym.com/clientportal2/?lang={it en}#/Classes/1/Calendar</code>	Group class booking calendar (<code>perfectGym.classBooking(lang)</code>)
PerfectGym — lead/contact record	(no direct URL — created by the n8n workflow behind <code>tca-form-compilato</code>)	Every submitted lead is created/updated as a contact in PerfectGym so staff manage it from the same platform used for members — see §13.1, §13.3
Playtomic	<code>https://playtomic.io/tennis-club-ambrosiano/...</code> ; <code>https://playtomic.com/clubs/club-ambrosiano-tennis</code>	Court booking (tennis/padel) (<code>playtomic.tennisCourt</code> / <code>.padelCourt</code> / <code>.club</code>)
Member app deep link	<code>https://onelink.to/7vfvz2</code>	App-store smart link for the member app (<code>memberApp.smartLink</code>)
Mini CRM — operational reporting	<code>https://crm.tcambrosiano.com</code>	Receives every submitted lead (forwarded by the n8n workflow, not called directly by the website) for internal operational reporting/tracking of contacts — see §13.3
TinaCloud	<code>app.tina.io</code>	Headless CMS auth/backend for the visual editor